PHOTON · AGENTS IN IMESSAGE

Snacxx uses Photon to the fullest.

One text to a friend in iMessage plans a meal for a whole group. Every Photon capability is in the product, each one doing real work, and it runs end to end on AWS.

9
Photon capabilities in production use

1 text
starts a private plan with every friend

~100
active users served in real time today

Serverless
pay per use, near-zero when idle

01 How Snacxx uses Photon

PHOTON FEATURE	WHAT IT DOES IN SNACXX	WHY IT MATTERS
Native polls	Multi-select time and place polls; the agent waits until taps settle and sends no reply per tap.	Friends vote with a tap and get no spam.
Find My	Requests and reads each person's location for the "closest fair spot" and nearby food.	A real, fair place instead of a guess.
Contact cards	A friend's shared contact becomes a registered user and gets an intro text.	Growth is one tap inside the chat.
Attachments	Reads food photos for calories; sends back rendered images.	Calorie tracking with no typing.
Voice notes	Voice in and out (ElevenLabs Scribe and Flash v2.5). Voice notes arrive without audio bytes, so the agent downloads them through Photon's API.	Hands-free, natural conversation.
Rich-link cards	Our two custom cards, built with the Spectrum library, arrive in the thread.	Product-grade UI inside a text.
Typing + reactions	Typing indicator while thinking; tapbacks on photos and contacts.	Reads like a friend, not a bot.
Webhooks	Every inbound message is delivered to our serverless backend, HMAC-signed.	Always on, scales with traffic.
Direct messages	Each friend is texted privately on their own line. No group chat is needed.	Group planning on Photon's shared pool.

02 Two cards, made with Spectrum

Today's Pie

A calorie, protein and monthly-budget tracker drawn as a pizza that fills as you eat, updated in place all day.

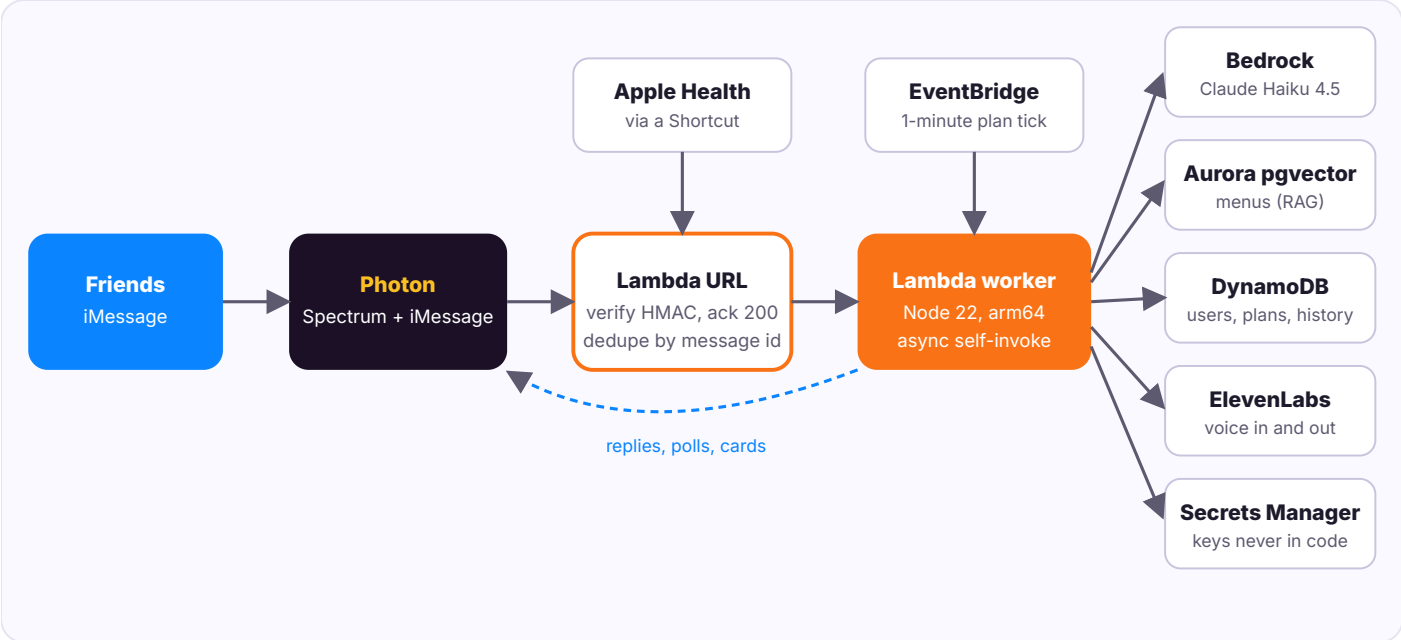
Event confirmation

Place, time, date, who's coming and each person's walk, sent to everyone once the group agrees.

ARCHITECTURE

From one text to a finished plan, on AWS

Serverless from edge to database: nothing to patch, nothing running when nobody is texting.



03 What happens to one message

1. **Photon delivers** the iMessage to our public Function URL as a signed webhook.
2. **The function verifies** the HMAC signature (5-minute replay window), acknowledges within milliseconds and drops duplicates.
3. **It hands off** to an async copy of itself, so slow work (the model, a long poll wait) never delays Photon. Retries are off, so a flow never runs twice.
4. **The worker thinks:** it loads the user and chat history, runs Claude on Bedrock with tools (location, menus, meal log, plan), and queries Aurora for menus.
5. **It answers through Photon:** text, poll, Find My request, voice note or rich card, and every friend is reached privately.
6. **EventBridge ticks each minute** to move group plans forward (time, then confirm, then place, then card) even while nobody is texting.

04 Deploy: one command, verified

Infrastructure as code

CloudFormation stacks create the function, tables, IAM, schedule and URL. Re-running is safe.

One-command deploy

`npm run deploy` builds, uploads, and we check the deployed code hash against the build. A self-test checks voice and Bedrock.

Observable

Every step logs to CloudWatch with the user, tool calls and replies, so a problem is traceable in seconds.

SCALE AND SECURITY

Built for 100 users now, and for growth

Every layer scales on demand. The limits below are settings and quotas, not rewrites.

05 Scale

LAYER	HOW IT SCALES	TODAY
Webhook + worker	Lambda runs one copy per message in parallel; 1.5 GB, arm64. Fast ack, slow work async.	**400 concurrent (adjustable)**
State	DynamoDB pay-per-request: no capacity to plan, scales to any request rate.	**Effectively unlimited**
Menu search	Aurora Serverless v2 with pgvector, over the Data API; scales from 0 up to its ACU cap.	**0 to 2 ACU (one setting to raise)**
Model	Bedrock Claude Haiku 4.5 through a cross-region inference profile.	50 requests/min default quota
Plans	EventBridge tick moves every active plan forward; plan state is stored per plan.	**Many plans in parallel**

Honest sizing: ~100 active users are comfortable today. The first limit we would meet is the model's requests-per-minute quota, which is a routine increase request. After that come the Aurora ACU cap and Lambda concurrency, both settings.

Growth path: 10x is three settings (Bedrock quota, Aurora ACU cap, reserved Lambda concurrency). 100x adds a queue (SQS) between webhook and worker and caching of menu answers. Each Photon user has their own line, so load also spreads across lines.

06 Security

CONCERN	WHAT WE DO
Who can call us	The URL is public because Photon can't sign AWS requests, so every delivery is authenticated by Photon's HMAC-SHA256 signature, checked in constant time with a 5-minute replay window. Unsigned requests are rejected.
Secrets	Photon and ElevenLabs keys live in Secrets Manager and are read at runtime; none are in code or the repo.
Least privilege	The function's IAM role can only touch its own tables, its own database cluster, its own secrets and itself.
Data at rest	Aurora storage is encrypted; the users table has encryption, point-in-time recovery and is retained if the stack is deleted; chat state expires on a TTL.
Food safety + privacy	A code-level guard, not the AI, checks allergies and diet, and the bot never calls a dish "safe". In groups, friends see only the merged restrictions needed to pick a place, never anyone's health numbers.
Consent + reliability	Only registered, opted-in users can message the bot, and invited friends can reply STOP. Duplicate deliveries are dropped, retries are off, and voice falls back to Polly if ElevenLabs is down.